SOLID Principi — Model klasa sistema

S — Single Responsibility Principle

Svaka klasa treba imati samo jedan razlog za promjenu.

Svaka klasa u modelu ima jasno definisanu i jedinstvenu odgovornost. Klasa `Korisnik` isključivo upravlja podacima o korisniku i autentifikacijom — ne sadrži logiku slanja notifikacija ni generisanja izvještaja. Klasa `Rezervacija` odgovorna je samo za životni ciklus rezervacije, dok je finansijska obrada odvojena u klasu Placanje. Klasa Notifikacija čuva podatkovni model jedne obavijesti, a slanje je delegirano interfejsu INotifikacioniServis i njegovoj implementaciji EmailNotifikacioniServis. Klasa Izvjestaj definira parametre izvještaja, dok generisanje dokumenta pripada klasi PdfIzvjestajGenerator. Klasa `MapaUpravljac` koordinira isključivo operacije prikaza mape, bez ikakve odgovornosti vezane za plaćanje ili autentifikaciju. Repozitorij interfejsi (IParkingRepozitorij, IRezervacijaRepozitorij, IPlacanjeRepozitorij, IKorisnikRepozitorij) svaki je zadužen za persistenciju tačno jednog entiteta. Value objekti GeografskeKoordinate, GeografskaOblast i ParkingFilter enkapsuliraju isključivo podatke vezane za svoju namjenu bez ikakvih dodatnih odgovornosti.

O — Open/Closed Principle

Klase trebaju biti otvorene za proširenje, a zatvorene za modifikaciju.

Model je projektovan tako da se nove funkcionalnosti mogu dodavati bez izmjene postojećeg koda. Uvođenje novog načina plaćanja (npr. PayPal ili kriptovalute) zahtijeva samo kreiranje nove klase koja implementira interfejs IPlacanje, bez ikakve izmjene postojeće klase `KarticaPlacanje` ili klase `Placanje`. Analogno, dodavanje novog formata izvještaja (npr. CSV ili Word) zahtijeva samo novu implementaciju interfejsa `IIzvjestajGenerator`, bez promjene klase `Izvjestaj` ili `PdfIzvjestajGenerator`. Dodavanje novog kanala notifikacija (npr. SMS) zahtijeva samo novu implementaciju `INotifikacioniServis`, bez modifikacije klase `EmailNotifikacioniServis`. Enumeracije `FormatIzvjestaja`, `TipNotifikacije`, `StatusRezervacije` i `UlogaKorisnika` jasno odvajaju moguće vrijednosti od poslovne logike, čime se proširenje vrši dodavanjem nove vrijednosti bez modifikacije klasa koje ih koriste.

L — Liskov Substitution Principle

Instanca implementacione klase mora biti upotrebljiva svuda gdje se koristi instanca interfejsa, bez narušavanja ispravnosti programa.

Svaka konkretna implementacija u modelu potpuno ispunjava ugovor definiran odgovarajućim interfejsom. Klasa `KarticaPlacanje` implementira sve metode interfejsa `IPlacanje` na način koji je funkcionalno ekvivalentan ugovoru — svaki pozivalac koji prima tip `IPlacanje` funkcionira identično bez obzira radi li sa `KarticaPlacanje` ili nekom budućom implementacijom. Klasa EmailNotifikacioniServis uvijek uspješno izvršava metodu posaljiNotifikaciju() bez bacanja neočekivanih iznimki, čime LSP nije narušen. Klasa `PdfIzvjestajGenerator` implementira metodu generiraj() i vraća rezultat tipa File kako interfejs `IIzvjestajGenerator` propisuje. Enumeracija `StatusRezervacije` sa vrijednostima `AKTIVNA`, `ISTEKLA` i `OTKAZANA` osigurava da svi mogući statusi budu konzistentno obrađivani bez posebnih provjera u pozivajućem kodu. Value objekt `GeografskeKoordinate` je neizmjenjiv — svaki poziv `toString()` vraća isti rezultat za iste ulazne vrijednosti, bez bočnih efekata.

I — Interface Segregation Principle

Klijenti ne smiju biti prisiljeni zavisiti od metoda koje ne koriste — interfejsi trebaju biti uski i specifični.

Svaki interfejs u modelu definira isključivo metode relevantne za jednu namjenu. `IPlacanje` sadrži samo metodu `obradiPlacanje()` i ne uključuje metode za generisanje izvještaja ili slanje notifikacija. `INotifikacioniServis` sadrži samo metodu `posaljiNotifikaciju()`. `IIzvjestajGenerator` sadrži samo metodu `generiraj()`. `IMapaServis` sadrži samo metodu `prikaziMarkere()`. Zahvaljujući ovakvoj podjeli, klasa `MapaUpravljac` zavisi samo od `IMapaServis` i `IParkingRepozitorij` te nije opterećena metodama vezanim za plaćanje ili notifikacije. Repozitorij interfejsi su međusobno odvojeni: `IParkingRepozitorij`, `IRezervacijaRepozitorij`, `IPlacanjeRepozitorij` i `IKorisnikRepozitorij` svaki eksponira samo metode vezane za upravljanje jednim entitetom. `IRezervacijaServis` je odvojen od `IRezervacijaRepozitorij` jer poslovna logika i persistencija predstavljaju različite odgovornosti.

D — Dependency Inversion Principle

Moduli visokog nivoa ne smiju zavisiti od modula niskog nivoa. Oba trebaju zavisiti od apstrakcija.

Cjelokupna arhitektura modela počiva na zavisnostima prema interfejsima, a ne prema konkretnim implementacijama. Klasa `MapaUpravljac` (modul visokog nivoa) ne zavisi od konkretne implementacije mape već od apstrakcije `IMapaServis` — konkretna implementacija ubrizgava se izvana. `MapaUpravljac` dohvaća podatke o parkingu putem `IParkingRepozitorij`, a ne direktnim pristupom bazi podataka. Klasa `EmailNotifikacioniServis` zavisi od `IKorisnikRepozitorij` za dohvatanje korisničkih podataka, a ne od konkretne implementacije repozitorija. Klasa `PdfIzvjestajGenerator` zavisi od repozitorij interfejsa (`IParkingRepozitorij`, `IRezervacijaRepozitorij`, `IPlacanjeRepozitorij`) kako bi prikupila potrebne podatke za generisanje izvještaja. Zahvaljujući ovom principu, moguće je koristiti in-memory implementacije svih interfejsa tokom testiranja bez pokretanja stvarne baze podataka ili eksternih servisa, što direktno poboljšava testabilnost i maintainability sistema.